



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
"МИРЭА — Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4
по дисциплине
«Разработка баз данных»

Выполнил студент группы ИНБО-20-23

Боргачев Т. М.

Принял доцент кафедры ЦТ

Исаева М.В.

Москва 2025

ПРАКТИЧЕСКАЯ РАБОТА №4. АНАЛИТИЧЕСКИЕ ЗАПРОСЫ: ОКОННЫЕ ФУНКЦИИ И ПОСТРОЕНИЕ СВОДНЫХ ТАБЛИЦ

Цель: формирование у студентов углубленных навыков работы со сложными аналитическими запросами в СУБД PostgreSQL.

Постановка задачи:

Для выполнения практической работы необходимо последовательно выполнить четыре задачи, используя собственную базу данных. Все примеры в данном документе основаны на демонстрационной базе данных «Аптека», содержащей таблицы `manufacturers` (производители), `medicines` (лекарства) и `sales` (продажи). Ваша задача — адаптировать каждую из поставленных задач к логической структуре и предметной области вашей базы данных. Приведенные ниже формулировки и последующие примеры кода служат шаблоном для понимания, какой тип аналитического запроса требуется составить.

Задание №1: использование ранжирующих функций. Для каждой основной «родительской» сущности в вашей БД (например, производитель, категория товара, автор) определить три наиболее значимых по некоторому числовому признаку дочерних сущности (например, три самых дорогих товара, три самые популярные книги по количеству продаж). В результирующей таблице должны быть указаны идентификатор группы, идентификатор дочерней сущности, её числовой признак и ранг. Для расчёта ранга использовать функцию `RANK()` или `DENSE_RANK()`.

Задание №2: использование агрегатных оконных функций. Для ключевой сущности, имеющей транзакции во времени (например, товар, услуга), рассчитать нарастающий итог (кумулятивную сумму) по некоторому показателю (например, объем продаж, количество заказов) с разбивкой по временным периодам (месяцам или годам). Отчёт должен содержать идентификатор сущности, временной период, сумму за период и кумулятивную сумму с начала наблюдений.

Задание №3: использование функции смещения. Провести сравнительный анализ общих показателей по периодам. Для каждого периода (например, месяца), начиная со второго, необходимо вывести общий показатель за текущий период и аналогичный показатель за предыдущий период в одной строке. Это позволит наглядно оценить динамику. Необходимо использовать функцию `LAG()`.

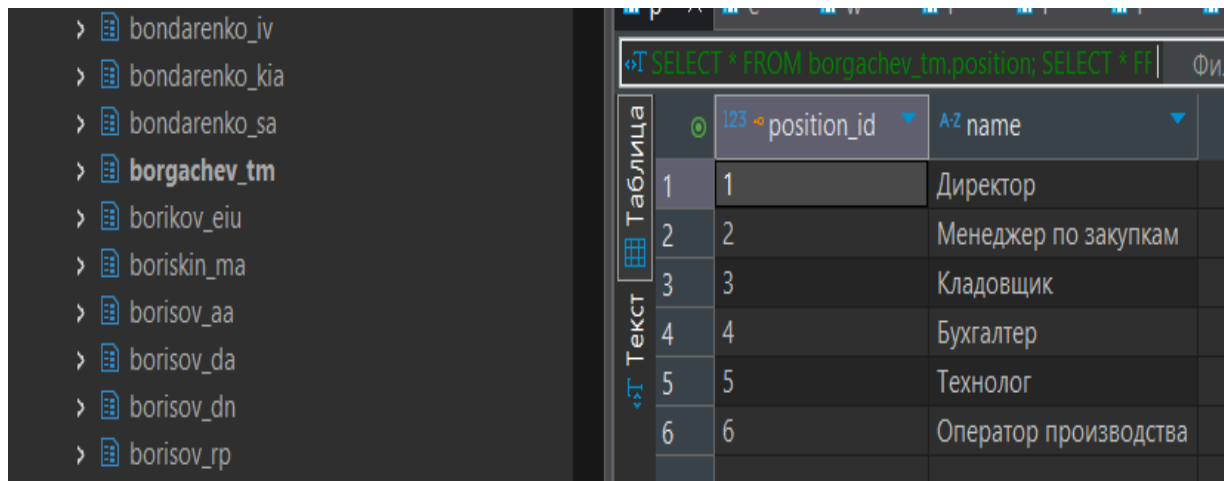
Задание №4: построение сводной таблицы. Создать сводный отчет, который агрегирует некоторый числовой показатель для основной сущности по категориям, представленным в виде столбцов. Например, показать общую сумму продаж для каждого

товара по кварталам года. Строки в отчете должны представлять основные сущности, а столбцы — категории. Задачу необходимо решить двумя способами: 1. С использованием условной агрегации (комбинация SUM и CASE). 2. С использованием функции crosstab из расширения tablefunc.

ХОД РАБОТЫ

Задание №1: использование ранжирующих функций

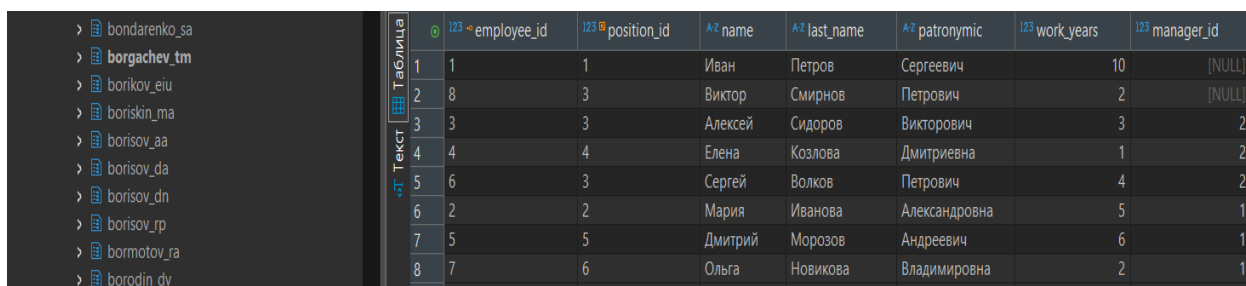
Таблица position представлена на Рисунке 1.



position_id	name
1	Директор
2	Менеджер по закупкам
3	Кладовщик
4	Бухгалтер
5	Технолог
6	Оператор производства

Рисунок 1 —Таблица position

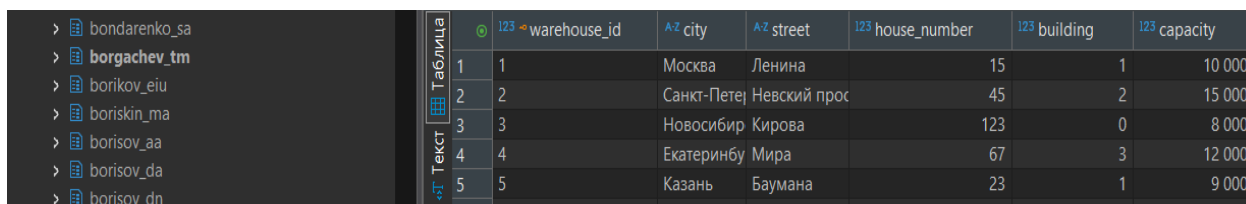
Таблица employee представлена на Рисунке 2.



employee_id	position_id	name	last_name	patronymic	work_years	manager_id
1	1	Иван	Петров	Сергеевич	10	[NULL]
2	8	Виктор	Смирнов	Петрович	2	[NULL]
3	3	Алексей	Сидоров	Викторович	3	2
4	4	Елена	Козлова	Дмитриевна	1	2
5	6	Сергей	Волков	Петрович	4	2
6	2	Мария	Иванова	Александровна	5	1
7	5	Дмитрий	Морозов	Андреевич	6	1
8	7	Ольга	Новикова	Владимировна	2	1

Рисунок 2 —Таблица employee

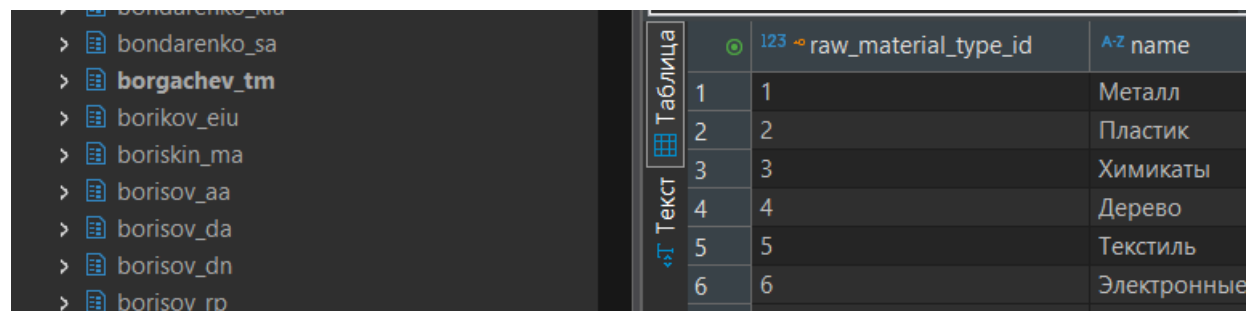
Таблица warehouse представлена на Рисунке 3.



warehouse_id	city	street	house_number	building	capacity
1	Москва	Ленина	15	1	10 000
2	Санкт-Петербург	Невский прот	45	2	15 000
3	Новосибирск	Кирова	123	0	8 000
4	Екатеринбург	Мира	67	3	12 000
5	Казань	Баумана	23	1	9 000

Рисунок 3 —Таблица warehouse

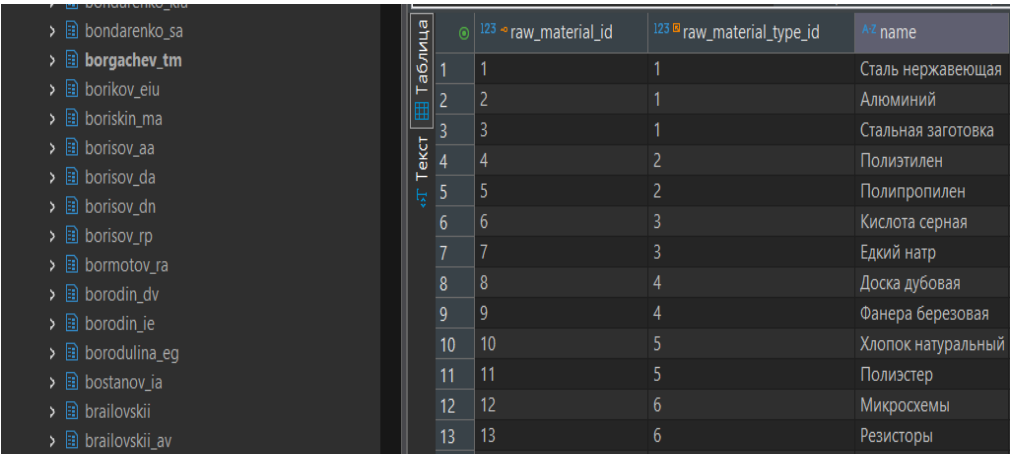
Таблица raw_material_type представлена на Рисунке 4.



raw_material_type_id	name
1	Металл
2	Пластик
3	Химикаты
4	Дерево
5	Текстиль
6	Электронные

Рисунок 4 —Таблица raw_material_type

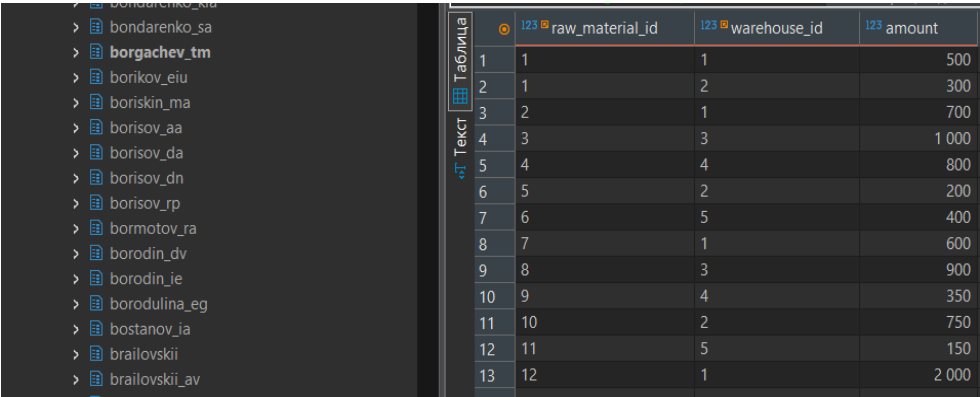
Таблица raw_material представлена на Рисунке 5.



	123 raw_material_id	123 raw_material_type_id	A-Z name
1	1	1	Сталь нержавеющая
2	2	1	Алюминий
3	3	1	Стальная заготовка
4	4	2	Полиэтилен
5	5	2	Полипропилен
6	6	3	Кислота серная
7	7	3	Едкий натр
8	8	4	Доска дубовая
9	9	4	Фанера березовая
10	10	5	Хлопок натуральный
11	11	5	Полиэстер
12	12	6	Микросхемы
13	13	6	Резисторы

Рисунок 5 —Таблица raw_material

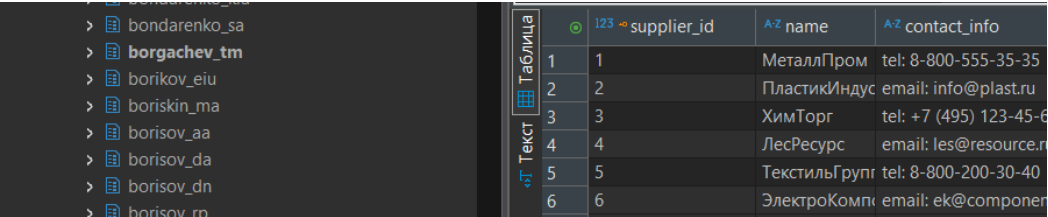
Таблица inventory представлена на Рисунке 6.



	123 raw_material_id	123 warehouse_id	123 amount
1	1	1	500
2	1	2	300
3	2	1	700
4	3	3	1 000
5	4	4	800
6	5	2	200
7	6	5	400
8	7	1	600
9	8	3	900
10	9	4	350
11	10	2	750
12	11	5	150
13	12	1	2 000

Рисунок 6 —Таблица inventory

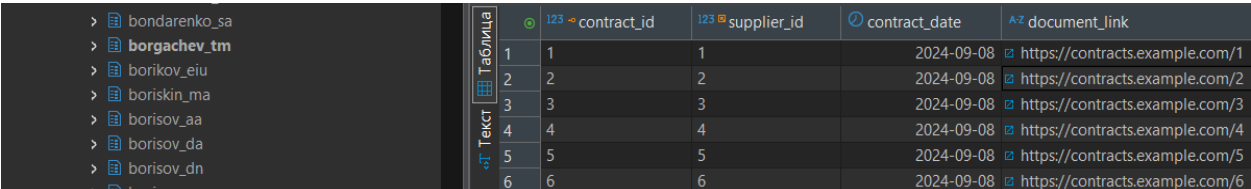
Таблица supplier представлена на Рисунке 7.



	123 supplier_id	A-Z name	A-Z contact_info
1	1	МеталлПром	tel: 8-800-555-35-35
2	2	ПластикИндус	email: info@plast.ru
3	3	ХимТорг	tel: +7 (495) 123-45-6
4	4	ЛесРесурс	email: les@resource.ru
5	5	ТекстильГрупп	tel: 8-800-200-30-40
6	6	ЭлектроКомп	email: ek@componen

Рисунок 7 —Таблица supplier

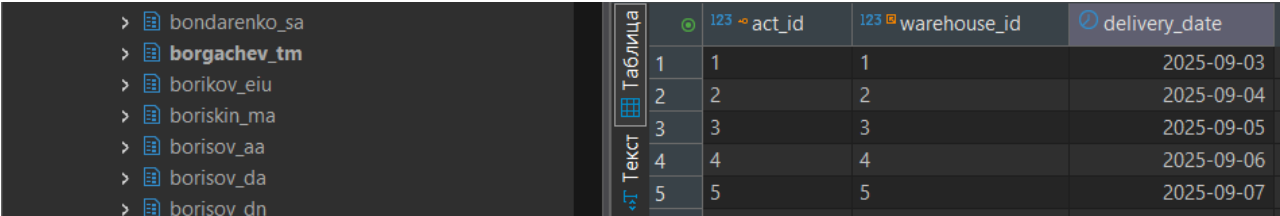
Таблица contract представлена на Рисунке 8.



	123 contract_id	123 supplier_id	contract_date	A-Z document_link
1	1	1	2024-09-08	https://contracts.example.com/1
2	2	2	2024-09-08	https://contracts.example.com/2
3	3	3	2024-09-08	https://contracts.example.com/3
4	4	4	2024-09-08	https://contracts.example.com/4
5	5	5	2024-09-08	https://contracts.example.com/5
6	6	6	2024-09-08	https://contracts.example.com/6

Рисунок 8 —Таблица contract

Таблица delivery_act представлена на Рисунке 9.



	123 act_id	123 warehouse_id	delivery_date
1	1	1	2025-09-03
2	2	2	2025-09-04
3	3	3	2025-09-05
4	4	4	2025-09-06
5	5	5	2025-09-07

Рисунок 9 —Таблица delivery_act

Таблица material_order представлена на Рисунке 10.

	order_id	raw_material_id	employee_id	supplier_id	contract_id	order_date
1	1	1	3	1	1	2025-09-03
2	2	2	3	1	1	2025-09-03
3	3	1	3	1	1	2025-09-03
4	4	2	3	1	1	2025-09-03
5	5	1	3	1	1	2025-09-03
6	6	2	3	1	1	2025-09-03
7	7	1	3	1	1	2025-09-07
8	8	1	3	1	1	2025-09-07
9	9	1	3	1	1	2025-09-07
10	10	1	3	1	1	2025-09-07
11	11	1	3	1	1	2025-09-07
12	34	1	3	1	1	2024-01-15
13	35	2	3	2	2	2024-01-20
14	36	3	2	3	3	2024-02-05
15	37	4	3	4	4	2024-02-10
16	38	5	2	5	5	2024-02-15
17	39	6	3	1	1	2024-03-01
18	40	7	2	2	2	2024-03-05
19	41	8	3	3	3	2024-03-10
20	42	9	2	4	4	2024-03-15
21	43	10	3	5	5	2024-04-01
22	44	11	2	1	1	2024-04-05
23	45	12	3	2	2	2024-04-10

Рисунок 10 —Таблица material_order

Таблица detail_type представлена на Рисунке 11.

detail_type_id	name
1	Металл
2	Пластик
3	Химикаты
4	Дерево
5	Текстиль
6	Электронные
7	Турбина
8	Сталь

Рисунок 11 —Таблица detail_type

Таблица detail представлена на Рисунке 12.

detail_id	detail_type_id	name
1	1	Деталь 1
2	2	Деталь 2
3	7	Турбина ГТД
4	8	Стальная деталь

Рисунок 12 —Таблица detail

Таблица detail_material представлена на Рисунке 13.

detail_id	raw_material_id	amount
4	1	10,5
1	1	5,2
1	2	8,7
2	4	12,3

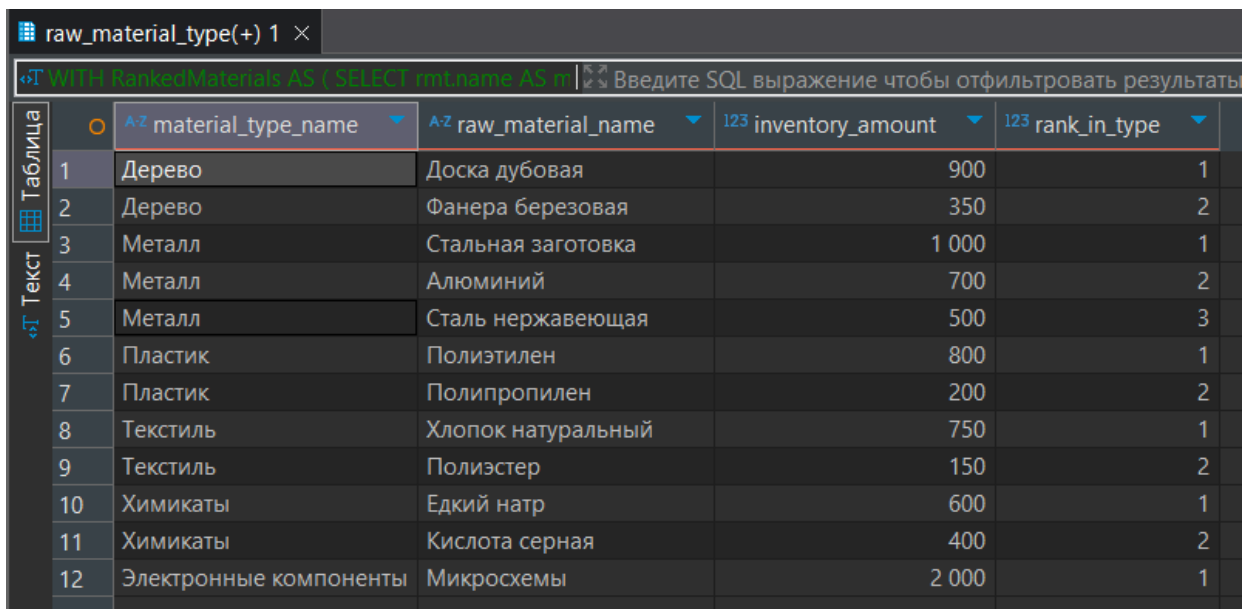
Рисунок 13 —Таблица detail_material

Запрос с использованием ранжирующих функций представлен в Листинге 1.

Листинг 1 — Запрос с ранжированием

```
-- Комментарий: Задание 1 - ранжирование материалов по количеству на складе в разрезе
типов материалов
-- Цель: Для каждого типа материала найти 3 материала с наибольшим количеством на
складах
-- Используем DENSE_RANK() для определения топ-3 материалов по количеству в каждом
типе
WITH RankedMaterials AS (
    -- Используем RANK() для определения топ-3 материалов по количеству в каждом типе
    SELECT rmt.name AS material_type_name,          -- Название типа материала (роди-
    тельская сущность)
           rm.name AS raw_material_name,           -- Название материала (дочерняя
    сущность)
           i.amount AS inventory_amount,           -- Количество материала на складе
           DENSE_RANK() OVER (PARTITION BY rmt.raw_material_type_id ORDER BY i.amount
DESC) AS rank_in_type -- Ранг в пределах типа
    FROM borgachev_tm.raw_material_type rmt
    JOIN borgachev_tm.raw_material rm ON rmt.raw_material_type_id = rm.raw_mate-
    rial_type_id
    JOIN borgachev_tm.inventory i ON rm.raw_material_id = i.raw_material_id
)
SELECT material_type_name,
       raw_material_name,
       inventory_amount,
       rank_in_type
FROM RankedMaterials
WHERE rank_in_type <= 3 -- Оставляем только топ-3 в каждом типе
ORDER BY material_type_name, rank_in_type;
```

Результат запроса представлен на Рисунке 14.



	material_type_name	raw_material_name	inventory_amount	rank_in_type
1	Дерево	Доска дубовая	900	1
2	Дерево	Фанера березовая	350	2
3	Металл	Стальная заготовка	1 000	1
4	Металл	Алюминий	700	2
5	Металл	Сталь нержавеющая	500	3
6	Пластик	Полиэтилен	800	1
7	Пластик	Полипропилен	200	2
8	Текстиль	Хлопок натуральный	750	1
9	Текстиль	Полиэстер	150	2
10	Химикаты	Едкий натр	600	1
11	Химикаты	Кислота серная	400	2
12	Электронные компоненты	Микросхемы	2 000	1

Рисунок 14 — Результат ранжирующего запроса

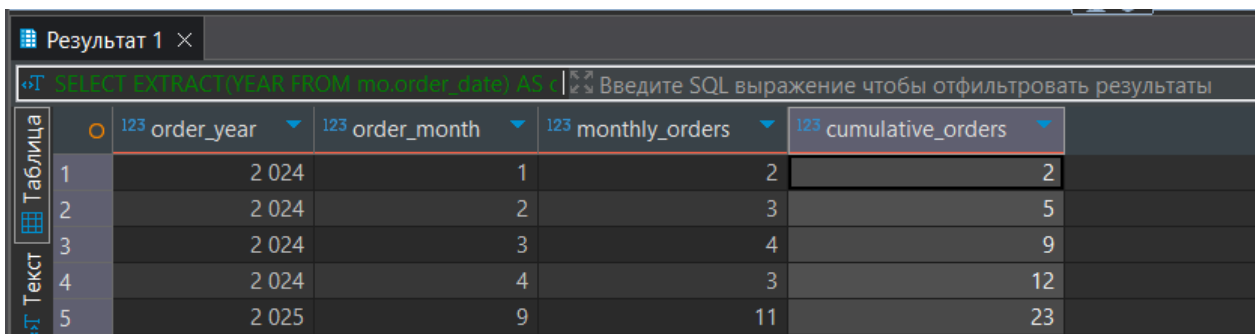
Задание №2: использование агрегатных оконных функций

Запрос с использованием агрегатных оконных функций представлен в Листинге 2.

Листинг 2 — Запрос с использованием агрегатных оконных функций

```
-- Комментарий: Задание 2 - расчет нарастающего итога заказов материалов по месяцам
-- Цель: Для каждого месяца показать количество заказов и кумулятивную сумму с начала
наблюдений
-- Используем SUM() OVER() для вычисления нарастающего итога
SELECT
    EXTRACT(YEAR FROM mo.order_date) AS order_year, -- Год заказа
    EXTRACT(MONTH FROM mo.order_date) AS order_month, -- Месяц заказа
    COUNT(*) AS monthly_orders, -- Количество заказов в месяце
    SUM(COUNT(*)) OVER (ORDER BY EXTRACT(YEAR FROM mo.order_date), EXTRACT(MONTH FROM
mo.order_date)) AS cumulative_orders -- Кумулятивное количество заказов
FROM borgachev_tm.material_order mo
GROUP BY EXTRACT(YEAR FROM mo.order_date), EXTRACT(MONTH FROM mo.order_date)
ORDER BY order_year, order_month;
```

Результат запроса представлен на Рисунке 15.



	123 order_year	123 order_month	123 monthly_orders	123 cumulative_orders
1	2 024	1	2	2
2	2 024	2	3	5
3	2 024	3	4	9
4	2 024	4	3	12
5	2 025	9	11	23

Рисунок 15 — Результат запроса с кумулятивной суммой

Задание №3: использование функции смещения

Запрос с использованием функции смещения представлен в Листинге 3.

Листинг 3 — Запрос с использованием функции смещения

```
-- Комментарий: Задание 3 - сравнение объема заказов по месяцам
-- Цель: Сравнить количество заказов текущего месяца с предыдущим месяцем
-- Используем LAG() для получения значения предыдущего периода
WITH monthly_orders AS (
    SELECT
        EXTRACT(YEAR FROM mo.order_date) AS order_year,
        EXTRACT(MONTH FROM mo.order_date) AS order_month,
        COUNT(*) AS orders_count
    FROM borgachev_tm.material_order mo
    GROUP BY EXTRACT(YEAR FROM mo.order_date), EXTRACT(MONTH FROM mo.order_date)
)
SELECT
    order_year,
    order_month,
    orders_count AS current_month_orders, -- Заказы текущего месяца
    LAG(orders_count) OVER (ORDER BY order_year, order_month) AS
previous_month_orders, -- Заказы предыдущего месяца
    orders_count - LAG(orders_count) OVER (ORDER BY order_year, order_month) AS
difference -- Разница между месяцами
FROM monthly_orders
ORDER BY order_year, order_month;
```

Результат запроса представлен на Рисунке 16.

Результат 1 ×					
WITH monthly_orders AS (SELECT EXTRACT(YEAR FROM mo.order_date) AS year, EXTRACT(QUARTER FROM mo.order_date) AS quarter, SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 1 THEN 1 ELSE 0 END) AS q1_orders, SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 2 THEN 1 ELSE 0 END) AS q2_orders, SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 3 THEN 1 ELSE 0 END) AS q3_orders, SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 4 THEN 1 ELSE 0 END) AS q4_orders FROM borgachev_tm.raw_material_type rmt JOIN borgachev_tm.raw_material rm ON rmt.raw_material_type_id = rm.raw_material_type_id JOIN borgachev_tm.material_order mo ON rm.raw_material_id = mo.raw_material_id WHERE EXTRACT(YEAR FROM mo.order_date) = 2024 GROUP BY rmt.raw_material_type_id, rmt.name ORDER BY rmt.name;					
Таблица	123 order_year	123 order_month	123 current_month_orders	123 previous_month_orders	123 difference
1	2 024	1	2	[NULL]	[NULL]
2	2 024	2	3	2	1
3	2 024	3	4	3	1
4	2 024	4	3	4	-1
5	2 025	9	11	3	8

Рисунок 16 — Результат запроса с LAG()

Задание №4: построение сводной таблицы

Запрос с построением сводной таблицы вариант 1 представлен в Листинге 4.

Листинг 4 — Вариант 1 сводной таблицы

```
-- Комментарий: Задание 4, вариант 1 - сводная таблица заказов по кварталам
-- Цель: Показать количество заказов для каждого типа материала по кварталам 2024
-- Используем условную агрегацию с CASE для транспонирования данных
SELECT
    rmt.name AS material_type, -- Тип материала (строки)
    SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 1 THEN 1 ELSE 0 END) AS
q1_orders, -- Заказы в Q1
    SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 2 THEN 1 ELSE 0 END) AS
q2_orders, -- Заказы в Q2
    SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 3 THEN 1 ELSE 0 END) AS
q3_orders, -- Заказы в Q3
    SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 4 THEN 1 ELSE 0 END) AS
q4_orders -- Заказы в Q4
FROM borgachev_tm.raw_material_type rmt
JOIN borgachev_tm.raw_material rm ON rmt.raw_material_type_id =
rm.raw_material_type_id
JOIN borgachev_tm.material_order mo ON rm.raw_material_id = mo.raw_material_id
WHERE EXTRACT(YEAR FROM mo.order_date) = 2024
GROUP BY rmt.raw_material_type_id, rmt.name
ORDER BY rmt.name;
```

Результат запроса представлен на Рисунке 17.

raw_material_type 1 ×						
SELECT rmt.name AS material_type, SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 1 THEN 1 ELSE 0 END) AS q1_orders, SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 2 THEN 1 ELSE 0 END) AS q2_orders, SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 3 THEN 1 ELSE 0 END) AS q3_orders, SUM(CASE WHEN EXTRACT(QUARTER FROM mo.order_date) = 4 THEN 1 ELSE 0 END) AS q4_orders FROM borgachev_tm.raw_material_type rmt JOIN borgachev_tm.raw_material rm ON rmt.raw_material_type_id = rm.raw_material_type_id JOIN borgachev_tm.material_order mo ON rm.raw_material_id = mo.raw_material_id WHERE EXTRACT(YEAR FROM mo.order_date) = 2024 GROUP BY rmt.raw_material_type_id, rmt.name ORDER BY rmt.name;						
Таблица	A-2 material_type	123 q1_orders	123 q2_orders	123 q3_orders	123 q4_orders	
1	Дерево	2	0	0	0	
2	Металл	3	0	0	0	
3	Пластик	2	0	0	0	
4	Текстиль	0	2	0	0	
5	Химикаты	2	0	0	0	
6	Электронные компоненты	0	1	0	0	

Рисунок 17 — Результат построения сводной таблицы (Вариант с CASE)

Запрос с построением сводной таблицы вариант 2 представлен в Листинге 5.

Листинг 5 — Сводная таблица Вариант 2

```
-- Комментарий: Задание 4, вариант 2 - сводная таблица заказов по кварталам с использованием crosstab
-- Цель: Тот же результат, что и в варианте 1, но с использованием функции crosstab
-- Предварительно устанавливаем расширение tablefunc, если оно не установлено

SELECT * FROM crosstab(
    'SELECT
        rmt.name AS material_type,
        ''Q'' || EXTRACT(QUARTER FROM mo.order_date) AS quarter,
        COUNT(*) AS order_count
    FROM borgachev_tm.raw_material_type rmt
    JOIN borgachev_tm.raw_material rm ON rmt.raw_material_type_id = rm.raw_material_id
    JOIN borgachev_tm.material_order mo ON rm.raw_material_id = mo.raw_material_id
    WHERE EXTRACT(YEAR FROM mo.order_date) = 2024
    GROUP BY rmt.name, EXTRACT(QUARTER FROM mo.order_date)
    ORDER BY 1, 2',
    'VALUES (''Q1''), (''Q2''), (''Q3''), (''Q4'')'
) AS ct (
    material_type TEXT,
    q1_orders BIGINT,
    q2_orders BIGINT,
    q3_orders BIGINT,
    q4_orders BIGINT
)
ORDER BY material_type;
```

Результат запроса представлен на Рисунке 18.

	material_type	q1_orders	q2_orders	q3_orders	q4_orders
1	Дерево	2	[NULL]	[NULL]	[NULL]
2	Металл	3	[NULL]	[NULL]	[NULL]
3	Пластик	2	[NULL]	[NULL]	[NULL]
4	Текстиль	[NULL]	2	[NULL]	[NULL]
5	Химикаты	2	[NULL]	[NULL]	[NULL]
6	Электронные компоненты	[NULL]	1	[NULL]	[NULL]

Рисунок 18 — Сводная таблица с помощью crosstab

Дополнительный запрос для демонстрации сложного анализа представлен в Листинге 6.

Листинг 6 — Дополнительный запрос

```
-- Комментарий: Дополнительный запрос для демонстрации сложного анализа
-- Топ-3 сотрудников по количеству заказов в каждом департаменте (позиции)

SELECT
    p.name AS position_name,
    e.name || ' ' || e.last_name AS employee_name,
    COUNT(mo.order_id) AS orders_count,
    RANK() OVER (PARTITION BY p.position_id ORDER BY COUNT(mo.order_id) DESC) AS
rank_in_position
FROM borgachev_tm.employee e
JOIN borgachev_tm.position p ON e.position_id = p.position_id
LEFT JOIN borgachev_tm.material_order mo ON e.employee_id = mo.employee_id
GROUP BY p.position_id, p.name, e.employee_id, e.name, e.last_name
HAVING COUNT(mo.order_id) > 0
ORDER BY p.position_id, rank_in_position;
```

Результат запроса представлен на Рисунке 19.

position 1 ×		«T SELECT p.name AS position_name, e.name ' ' e. Введите SQL выражение чтобы отфильтровать результ		
	A-Z position_name ▼	A-Z employee_name ▼	123 orders_count ▼	123 rank_in_position ▼
1	Менеджер по закупкам	Мария Иванова	5	1
2	Кладовщик	Алексей Сидоров	18	1

Рисунок 19 — Результат дополнительного запроса